

Python Mock Test Report

Data Types: String, List, Tuple, Dict, Set — Methods, Indexing, Slicing, Errors

Total Questions	Correct	Partially Correct	Incorrect
34	26	2	6

STRING

Q1. x = "be active"; print(x[-6]) -> ?

Your Answer: a

Result: Correct | Correct Answer: 'a'

Explanation: length=9, negative index -6 maps to positive index 3 -> 'a'

Q2. x = "be active in class"; print(x[3:9]) -> ?

Your Answer: active

Result: Correct | Correct Answer: 'active'

Explanation: slicing [start:end] includes start, excludes end -> indices 3-8 spell "active"

Q3. x = "be active in class"; print(x[-6:-1:-1]) -> ?

Your Answer: ssalc

Result: Incorrect | Correct Answer: " (empty string)

Explanation: with negative step, start position (13) must be > end position (18) to get output; here it's not, so slice is empty

Q4. x = "good morning"; print(x.count("o",4,7)) -> ?

Your Answer: 1

Result: Correct | Correct Answer: 1

Explanation: substring range [4:7) = " mo", only one 'o'

Q5. a = "yellow"; print(a.replace("l","*",1)) -> ?

Your Answer: ye*low

Result: Correct | Correct Answer: 'ye*low'

Explanation: count=1 limits replacement to first occurrence only

Q6. x = "python3"; print(x.isalnum()) -> ?

Your Answer: True

Result: Correct | Correct Answer: True

Explanation: isalnum() is True if all chars are letters/digits with no space/special chars

Q7. x = "hello"; print(x[10]) -> ?

Your Answer: Index Error

Result: Correct | Correct Answer: IndexError: string index out of range

Explanation: strings support only indices within their length (0-4 here); accessing beyond raises IndexError

Q8. a = "hi good night"; print(a.title()) -> ?

Your Answer: Hi Good Night

Result: Correct | Correct Answer: 'Hi Good Night'

Explanation: title() capitalizes first letter of each word

LIST

Q9. l = [4, 'active', 'hello']; l.append([45.5, 90]); print(l) -> ?

Your Answer: None

Result: Incorrect | Correct Answer: [4, 'active', 'hello', [45.5, 90]]

Explanation: append() adds the argument as ONE single element (even a list stays nested); append() itself returns None but here we print l, not the return value

Q10. a = [1,2,3]; a.extend(45) -> ?

Your Answer: type error

Result: Correct | Correct Answer: TypeError: 'int' object is not iterable

Explanation: extend() only accepts iterables; a plain int is not iterable

Q11. a = [1,2,3]; a.extend("py"); print(a) -> ?

Your Answer: [1, 2, 3, "py"]

Result: Incorrect | Correct Answer: [1, 2, 3, 'p', 'y']

Explanation: extend() unpacks an iterable element by element; a string is iterated char by char, unlike append() which adds as one nested unit

Q12. n = [12,100,300]; n.insert(1,"python"); print(n) -> ?

Your Answer: [12, "python", 100, 300]

Result: Correct | Correct Answer: [12, 'python', 100, 300]

Explanation: insert(index,value) places value at given index, shifting rest right

Q13. l = ["hello","good","morning","people"]; print(l.pop(1)); print(l) -> ?

Your Answer: 'good' (partial - only first line answered)

Result: Partially Correct | Correct Answer: 'good' then ['hello', 'morning', 'people']

Explanation: pop(index) removes & returns element at that index; remaining list shifts

Q14. l = ["hello","good","morning"]; l.remove() -> ?

Your Answer: Type error, position is mandatory

Result: Correct | Correct Answer: TypeError: list.remove() takes exactly one argument (0 given)

Explanation: remove() requires exactly one argument (the value to remove); calling with none raises TypeError

Q15. l = ["hello","good","morning"]; l.remove("afternoon") -> ?

Your Answer: Value error

Result: Correct | Correct Answer: ValueError: list.remove(x): x not in list

Explanation: remove() raises ValueError if the value isn't found in the list

TUPLE

Q16. t = ("hello","hoy",12.2,42,"good",45,[12,45,75]); print(t[6][1]) -> ?

Your Answer: 45

Result: Correct | Correct Answer: 45

Explanation: t[6] gets nested list [12,45,75], then [1] gets 45

Q17. t = (45,42,78,12,42,45,12,45,75); print(t.index(45,3,8)) -> ?

Your Answer: 5

Result: Correct | Correct Answer: 5

Explanation: index(value,start,end) searches only within given range; first match of 45 in range [3,8) is at index 5

Q18. t = (45,42,78,12,42,45,12,45,75); print(t.index(100)) -> ?

Your Answer: index error

Result: Incorrect | Correct Answer: ValueError: tuple.index(x): x not in tuple

Explanation: IndexError = invalid position access; ValueError = value not found via .index()/remove(). Since 100 isn't in the tuple, it's ValueError

Q19. t = ("hello","hoy",12.2,42,"good",45,(12,45,75)); print(t[-7:-5:1]) -> ?

Your Answer: "hello", "hoy"

Result: Correct | Correct Answer: ('hello', 'hoy')

Explanation: length=7; -7 maps to index 0, -5 maps to index 2; slice [0:2] gives ('hello','hoy')

DICT

Q20. food = {'idli':25,'dosa':45}; print(food.get("upma","not available")) -> ?

Your Answer: not available

Result: Correct | Correct Answer: 'not available'

Explanation: get(key, default) returns the default value if the key doesn't exist, instead of raising KeyError

Q21. food = {'idli':25,'dosa':45}; print(food["upma"]) -> ?

Your Answer: key error

Result: Correct | Correct Answer: KeyError: 'upma'

Explanation: direct dict indexing with [] raises KeyError when the key doesn't exist

Q22. p = {"banglore":576001}; p.setdefault("kalaburagi"); print(p) -> ?

Your Answer: {"banglore": 576001,"kalaburagi":None}

Result: Correct | Correct Answer: {'banglore': 576001, 'kalaburagi': None}

Explanation: setdefault(key) with no value inserts key with default value None

Q23. a={1:2}; a.update({"a":9}); a.update({(90,67):[1,2,3]}); print(a) -> ?

Your Answer: {1: 2, "a": 9, (90, 67): [1, 2, 3]}

Result: Correct | Correct Answer: {1: 2, 'a': 9, (90, 67): [1, 2, 3]}

Explanation: update() merges key-value pairs; tuples (immutable) can be dict keys, lists cannot

SET

Q24. s = {1,2,3}; print(s[1]) -> ?

Your Answer: Type error

Result: Correct | Correct Answer: TypeError: 'set' object is not subscriptable

Explanation: sets are unordered, unindexed collections -> no [] indexing support

Q25. s = {1,"instagram","whatsapp"}; s.discard("zoo") -> ?

Your Answer: Type error

Result: Incorrect | Correct Answer: No error - discard() silently does nothing if item not found

Explanation: discard() never raises an error for missing items; remove() would raise KeyError instead - this is the key difference between the two

Q26. `s = {1,"instagram","whatsapp"}; s.remove("INSTAGRAM") -> ?`

Your Answer: key error

Result: Correct | Correct Answer: `KeyError: 'INSTAGRAM'`

Explanation: remove() raises KeyError for missing/mismatched items (case-sensitive)

Q27. `s={1,2,3}; s1={7,5,9,6}; print(s.union(s1)); print(s) -> ?`

Your Answer: {1,2,3,7,5,6,9} for both lines

Result: Partially Correct | Correct Answer: {1,2,3,5,6,7,9} then {1,2,3} (s unchanged)

Explanation: union() returns a new set and does NOT modify the original; update() is the one that modifies in place

Q28. `a={"facebook","instagram","snapchat"}; a.add("facebook"); print(len(a)) -> ?`

Your Answer: 3 (plus set content shown)

Result: Correct | Correct Answer: 3

Explanation: add() does nothing if the item already exists in the set (no duplicates allowed)

Q29. `a={"good","morning","guys","how","are","you"}; b={"are","good","guys","you"}; print(b.issubset(a)); print(a.issubset(b)) -> ?`

Your Answer: true, false

Result: Correct | Correct Answer: True then False

Explanation: issubset checks if ALL items of the calling set exist in the other set; b's items all exist in a (True), but a has extra items not in b (False)

Q30. `a={"hello","hai",(1,2,3)}; b={"Hello","hal",(44,26,35)}; print(a.isdisjoint(b)) -> ?`

Your Answer: true

Result: Correct | Correct Answer: True

Explanation: strings are case-sensitive, so no elements truly match between sets -> isdisjoint returns True (no common elements)

TYPE CASTING / MISC

Q31. `a = "123"; print(int(a) + 5) -> ?`

Your Answer: 126

Result: Incorrect | Correct Answer: 128

Explanation: int("123") converts string to integer 123, then +5 = 128 (simple arithmetic slip)

LIST (contd)

Q32. `l=[10,20,30,40,50,60]; print(l[::2]) -> ?`

Your Answer: [10,30,50]

Result: Correct | Correct Answer: [10, 30, 50]

Explanation: step value 2 skips every other element starting from index 0

Q33. `l=[10,20,30,40,50]; print(l[::-1]) -> ?`

Your Answer: [50,40,30,20,10]

Result: Correct | Correct Answer: [50, 40, 30, 20, 10]

Explanation: step -1 with no start/end reverses the entire list

Q34. `b=[11,12,13,["a","b","c"]]; v=b.copy(); b[3][0]=77; print(v) -> ?`

Your Answer: [11, 12, 13, [77, "b", "c"]]

Result: Correct | Correct Answer: [11, 12, 13, [77, 'b', 'c']]

Explanation: copy() is a SHALLOW copy - the outer list is a new object, but nested/inner lists still share the same memory address. Changing a nested element via b also affects v.
